



南京大學
NANJING UNIVERSITY

九、模数转换器 (ADC)

主讲教师：闫浩
机器人与自动化学院



目录

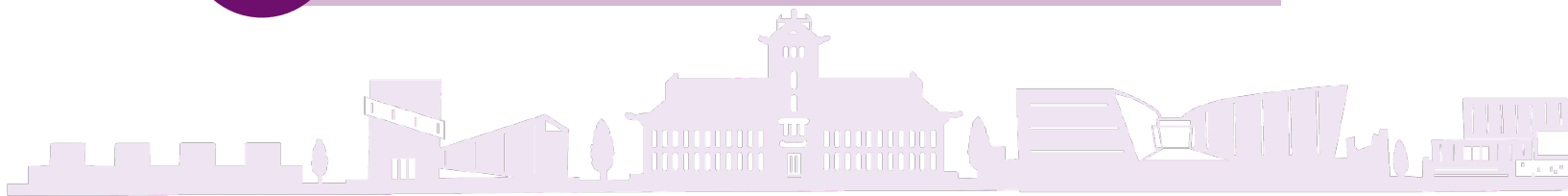
CONTENTS

01 ADC简介

02 STM32的ADC结构

03 STM32的ADC功能

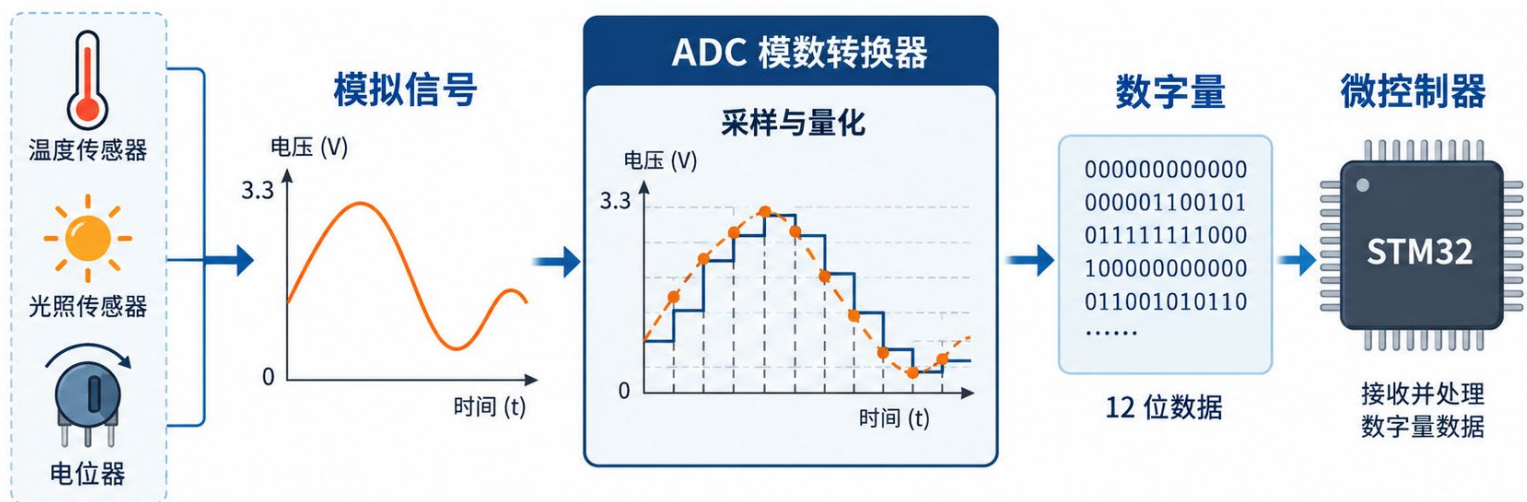
04 应用步骤及常用库函数



01 ADC简介

什么是ADC? ? ?

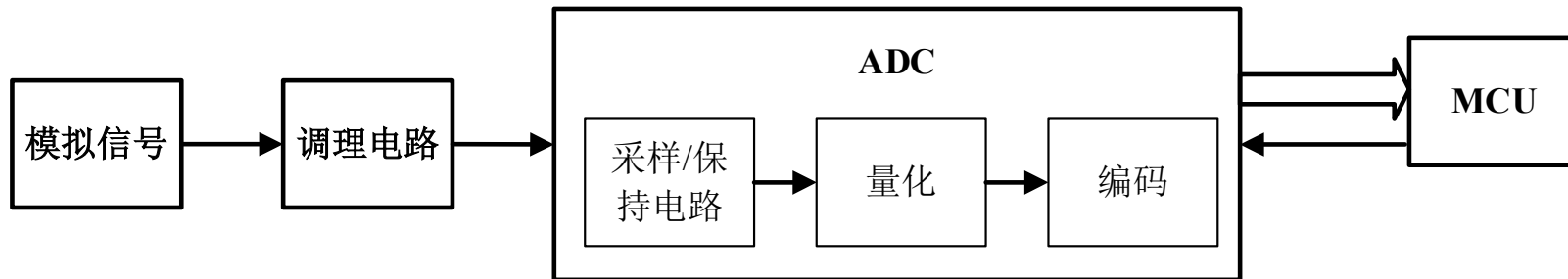
ADC (Analog-to-Digital Converter) 叫作模数转换器，作用是把连续变化的模拟电压转换为微控制器能够处理的数字量。



以 12位ADC、参考电压3.3 V 为例:

- 输出范围: 0 ~ 4095
- 输入0 V → 数字量约为0
- 输入1.65 V → 数字量约为2048
- 输入3.3 V → 数字量约为4095





1、ADC(Analog-to-Digital Converter):模/数转换器或者模拟/数字转换器。是指将**连续变量的模拟信号**转换为**离散的数字信号**的器件。

典型的模拟数字转换器将模拟信号转换为表示一定比例电压值的数字信号。

2 、在进行模/数转换时，要按**一定的时间间隔**，对模拟信号进行**采样**，然后再把采样得到的值**转换为数字量**。因此，模/数转换的基本过程由①**采样**、②**保持**、③**量化**和④**编码**组成。通常，采样和保持两个过程由采样-保持电路完成，量化和编码又常在转换过程中同时实现。

第一类为传感器输出的信号，如：

- (1) 电压信号：一般为mV信号，如热电偶（TC）的输出或电桥输出。
- (2) 电阻信号：单位为 Ω ，如热电阻（RTD）信号，通过电桥转换成mV信号。
- (3) 电流信号：一般为 μA 信号，如电流型集成温度传感器AD590的输出信号，通过取样电阻转换成mV信号。

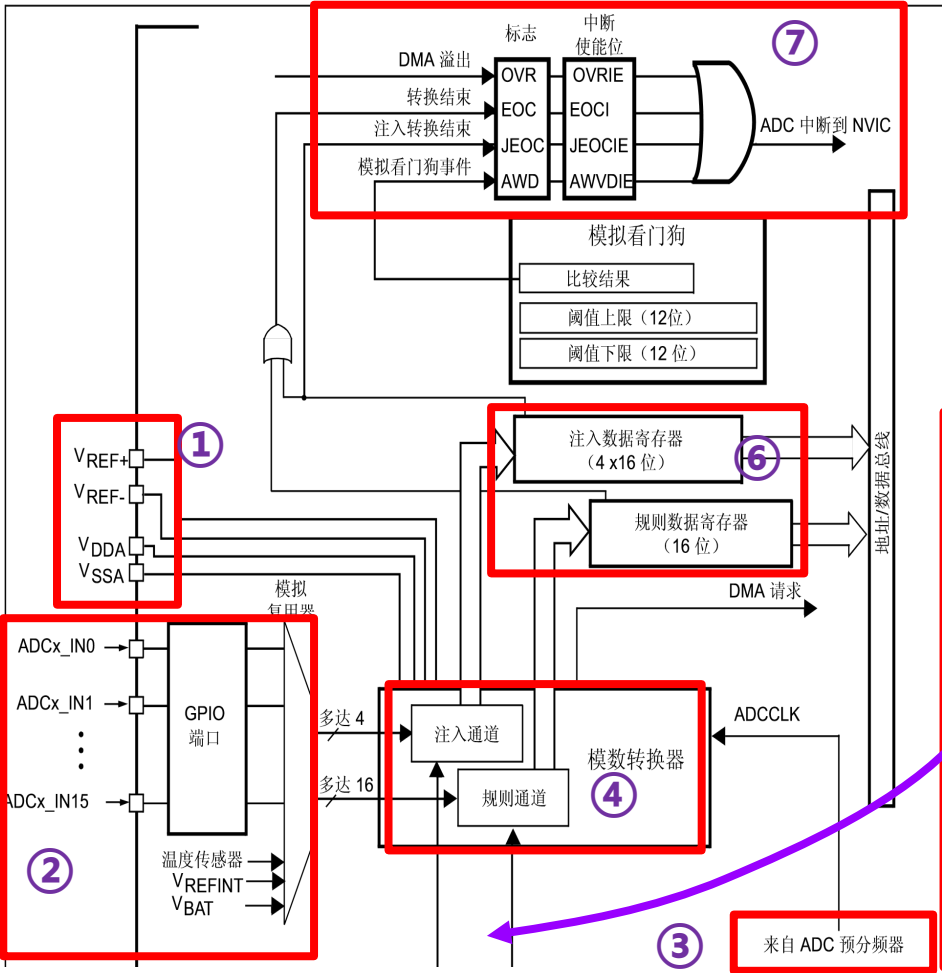
对于以上这些信号往往不能直接送A-D转换，因为信号的幅值太小，需经运算放大器放大后，变换成标准电压信号，如0~5V，1~5V，0~10V，-5V~+5V等，送往A-D转换器进行采样。有些双积分A-D转换器的输入为-200mV~+200mV或-2V~+2V，有些A-D转换器内部带有程控增益放大器（PGA），可直接接受mV信号。

第二类为变送器输出的信号，如：

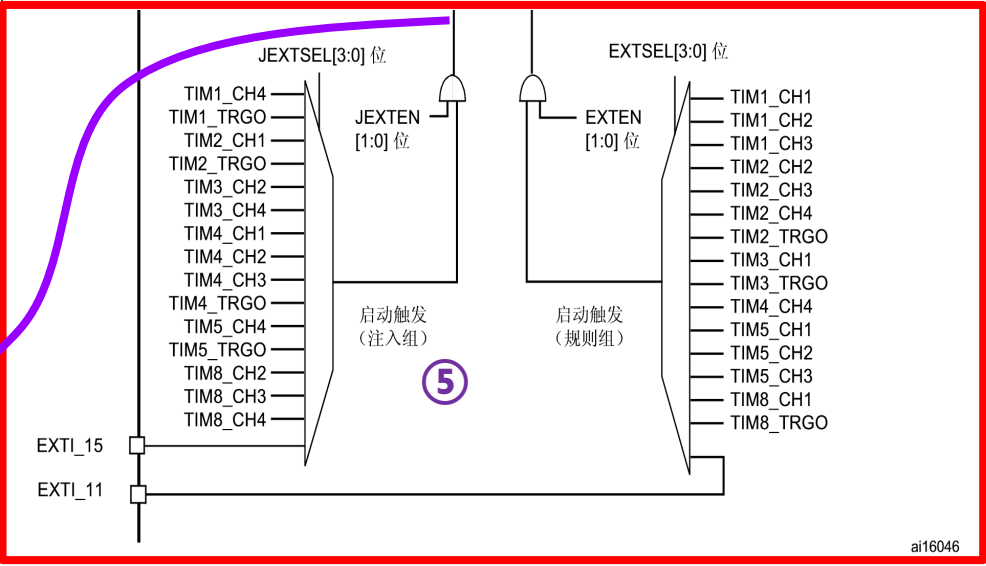
- (1) 电流信号：0~10mA（0~1.5k Ω 负载）或4~20mA（0~500 Ω 负载）。
- (2) 电压信号：0~5V或1~5V等。

电流信号可以远传，通过一个标准精密取样电阻就可以变成标准电压信号，送往A-D转换器进行采样，这类信号一般不需要放大处理。

02 STM32的ADC结构



- ①参考电压引脚
- ②模拟电压输入引脚
- ③ADC转换时钟源
- ④ADC转换通道
- ⑤ADC转换触发源
- ⑥ADC转换结果存储寄存器
- ⑦中断输出



1. 电源引脚

名称	信号类型	备注
V_{REF+}	正模拟参考电压输入	ADC 高/正参考电压， $1.8\text{ V} \leq V_{REF+} \leq V_{DDA}$
V_{DDA}	模拟电源输入	模拟电源电压等于 V_{DD} ， 全速运行时， $2.4\text{ V} \leq V_{DDA} \leq V_{DD} (3.6\text{ V})$ ； 低速运行时， $1.8\text{ V} \leq V_{DDA} \leq V_{DD}$
V_{REF-}	负模拟参考电压输入	ADC 低/负参考电压， $V_{REF-} = V_{SSA}$
V_{SSA}	模拟电源接地输入	模拟电源接地电压等于 V_{SS}
<u>$ADCx_IN[15:0]$</u>	模拟输入信号	<u>16 个模拟输入通道</u>

2. 输入引脚

STM32F4 ADC IO 分配					
ADC1	IO	ADC2	IO	ADC3	IO
通道0	PA0	通道0	PA0	通道0	PA0
通道1	PA1	通道1	PA1	通道1	PA1
通道2	PA2	通道2	PA2	通道2	PA2
通道3	PA3	通道3	PA3	通道3	PA3
通道4	PA4	通道4	PA4	通道4	PF6
通道5	PA5	通道5	PA5	通道5	PF7
通道6	PA6	通道6	PA6	通道6	PF8
通道7	PA7	通道7	PA7	通道7	PF9
通道8	PB0	通道8	PB0	通道8	PF10
通道9	PB1	通道9	PB1	通道9	PF3
通道10	PC0	通道10	PC0	通道10	PC0
通道11	PC1	通道11	PC1	通道11	PC1
通道12	PC2	通道12	PC2	通道12	PC2
通道13	PC3	通道13	PC3	通道13	PC3
通道14	PC4	通道14	PC4	通道14	PF4
通道15	PC5	通道15	PC5	通道15	PF5
通道16	连接内部VSS	通道16	连接内部VSS	通道16	连接内部VSS
通道17	连接内部Vrefint	通道17	连接内部VSS	通道17	连接内部VSS
通道18	连接内部温度传感器/内部VBAT	通道18	连接内部VSS	通道18	连接内部VSS

3. ADC转换时钟源

STM32F4系列微控制器的ADC是逐次比较逼近型，因此必须使用**驱动时钟**。所有ADC共用时钟ADCCLK，它来自经可编程预分频器分频的APB2时钟，该预分频器允许ADC在 $f_{PCLK2}/2$ 、 $f_{PCLK2}/4$ 、 $f_{PCLK2}/6$ 或 $f_{PCLK2}/8$ 等频率下工作。

ADCCLK最大频率为36MHz。

4. ADC转换通道

① **规则通道组**：相当正常运行的程序。最多16个通道。

规则通道和它的转换顺序在ADC_SQRx寄存器中选择，规则组转换的总数 写入ADC_SQR1寄存器的L[3:0]中

规则通道组共用一个转换数据寄存器！！

ADC_SQRx 规则序列寄存器 设置序列长度以及转换通道

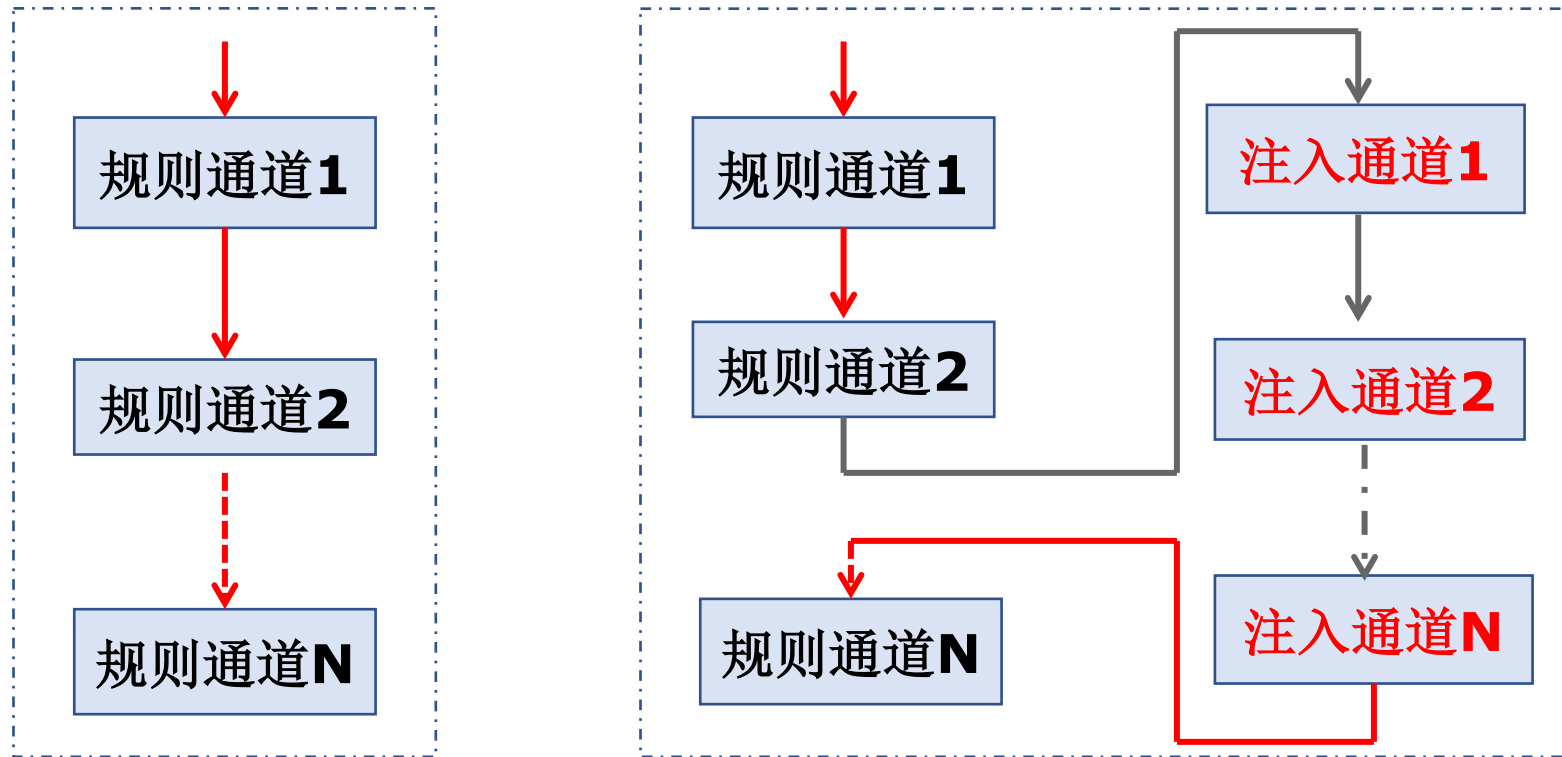
② **注入通道组**：相当于“中断”。最多4个通道。

注入组和它的转换顺序在ADC_JSQR寄存器中选择。注入组里转换的总数应写入ADC_JSQR寄存器的JL[1:0]中

每个注入通道都有一个独立的转换数据寄存器！！

ADC_JSQR 注入序列寄存器 设置注入通道及转换顺序

4. ADC转换通道



STM32F4的ADC的各通道可以①**单次**，②**连续**，③**扫描**或者④**间断模式**执行。

5. ADC转换触发源

1)、软件触发：ADC **使能**可以由 **ADC 控制寄存器 2: ADC_CR2 的ADON** 这个位来控制，写 1 的时候开始转换，写 0 的时候 停止转换，这个是最简单也是最好理解的开启 ADC 转换的控制方式。

另：(1) 位SWSTART：启动规则组。

(2) 位JSWSTART：启动注入组

2)、外部事件触发转换：触发源有很多，具体选择哪一种触发源，由 **ADC 控制寄存器2:ADC_CR2 的EXTSEL[2:0]和JEXTSEL[2:0]位来控制。**

(1) EXTSEL[2:0]用于选择规则通道的触发源；

(2) JEXTSEL[2:0]用于选择注入通道的触发源。

6. ADC转换结果存储寄存器

1)、**注入组有四个转换结果寄存器 (ADC_JDRx, 注入数据寄存器)** , 分别对应于每一个注入组通道。

2)、**而规则组只有一个数据寄存器 (ADC_DR, 规则数据寄存器)** , 所有规则组通道转换结果共用一个这个寄存器, 因此, 在使用规则组转换多路模拟信号的时, 多使用DMA配合。

03 STM32的ADC功能

1. ADC的使能与启动

单次转换模式下，**ADC只执行一次转换**。该模式既可通过设置ADC_CR2寄存器的**CONT位为0**。

一旦选择通道的转换完成：

- **如果一个规则通道被转换：**

- 转换数据被储存在16位ADC_DR寄存器中
- EOC(转换结束)标志被设置
- 如果设置了EOCIE，则产生中断。

- **如果一个注入通道被转换：**

- 转换数据被储存在16位的ADC_JDR1寄存器中
- JEOC(注入转换结束)标志被设置
- 如果设置了JEOCIE位，则产生中断。

然后ADC停止。

2. 时钟配置

ADC 挂载在APB2

1) 、ADC 的总转换时间跟 ADC 的输入时钟和采样时间有关，公式为：

$$T_{conv} = \text{采样时间} + 12 \text{ 个周期}$$

2) 、当 $ADCCLK = 30MHz$ ，即 $PCLK2$ 为 $60MHz$ ，ADC 时钟为 2 分频，采样时间设置为 3 个周期，那么总的转换时为： $f_{PCLK2}/2$ 、/4、/6 或 /8

$$T_{conv} = 3 + 12 = 16 \text{ 个周期} = 0.5\mu s。$$

3) 、通常， $f_{PCLK2} = 90MHz$ ，经过 ADC 预分频器能分频到最大的时钟只能是 $22.5M$ ，采样周期设置为 3 个周期，算出最短的转换时间为 $0.6667\mu s$ 。

ADC完整转换时间
跟ADC位数有关系

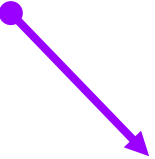
12 位: $3 + 12 = 15$ ADCCLK 周期

10 位: $3 + 10 = 13$ ADCCLK 周期

8 位: $3 + 8 = 11$ ADCCLK 周期

6 位: $3 + 6 = 9$ ADCCLK 周期

36MHz



3. 转换模式

(1) 单次方式

单次转换模式下，**ADC只执行一次转换**。该模式既可通过设置ADC_CR2寄存器的**CONT**位为0。

一旦选择通道的转换完成：

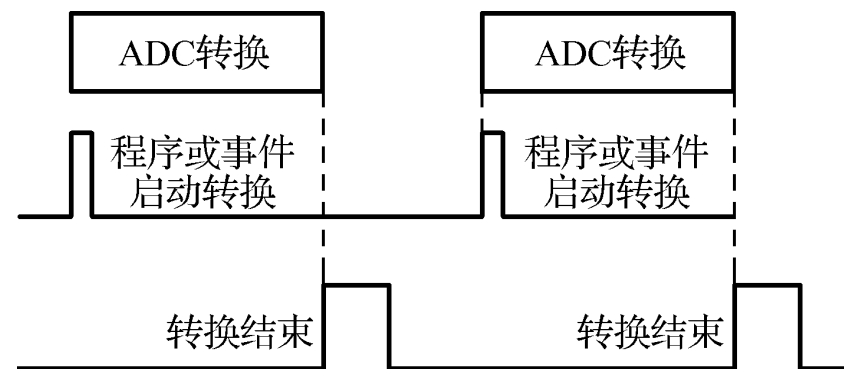
- 如果一个规则通道被转换：**

- 转换数据被储存在16位ADC_DR寄存器中
- EOC(转换结束)标志被设置
- 如果设置了EOCIE，则产生中断。

- 如果一个注入通道被转换：**

- 转换数据被储存在16位的ADC_JDR1寄存器中
- JEOC(注入转换结束)标志被设置
- 如果设置了JEOCIE位，则产生中断。

然后ADC停止。



3. 转换模式

(2) 连续方式

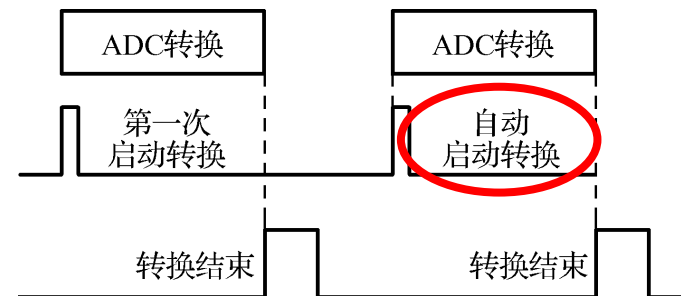
在连续转换模式中，**当前面ADC转换一结束马上就启动另一次转换。此时ADC_CR2寄存器CONT位是1。**
每个转换后：

- **如果一个规则通道被转换：**

- 转换数据被储存在16位的ADC_DR寄存器中
- EOC(转换结束)标志被设置
- 如果设置了EOCIE，则产生中断。

- **如果一个注入通道被转换：**

- 转换数据被储存在16位的ADC_DRJ1寄存器中
- JEOC(注入转换结束)标志被设置
- 如果设置了JEOCIE位，则产生中断。



3. 转换模式

(3) 扫描方式



- 扫描一组模拟通道。设置ADC_CR1寄存器的SCAN位为 '1' 来选择。
- ADC扫描所有被ADC_SQRX寄存器(对规则通道)或ADC_JSQR(对注入通道)选中的所有通道。
在每个组的每个通道上执行单次转换。
- 在每个转换结束时，同一组的下一个通道被自动转换。
- ADC_CR2寄存器CONT位，转换不会在选择组的最后一个通道上停止，而是再次从选择组的第一个通道继续转换。
- 如果设置了DMA位，在每次EOC后，DMA控制器把规则组通道的转换数据传输到SRAM中。
规则组扫描方式转换一般要结合DMA一起使用。
- 注入通道转换的数据总是存储在ADC_JDRx寄存器中。

3. 转换模式

(4) 间断方式

间断模式用来执行一个短序列的 n 次转换 ($n \leq 8$)，此转换是ADC_SQR $_x$ 所选择的转换序列的一部分。数值 n 由ADC_CR1的DISCNUM[2:0]位给出。

设置 $n=3$ ，总的序列长度=8，被转换的通道= 0、1、2、3、6、7、9、10，在间断模式下，转换的过程如下。

第一次触发：转换的序列为0、1、2。

第二次触发：转换的序列为3、6、7。

第三次触发：转换的序列为9、10，并产生EOC事件。

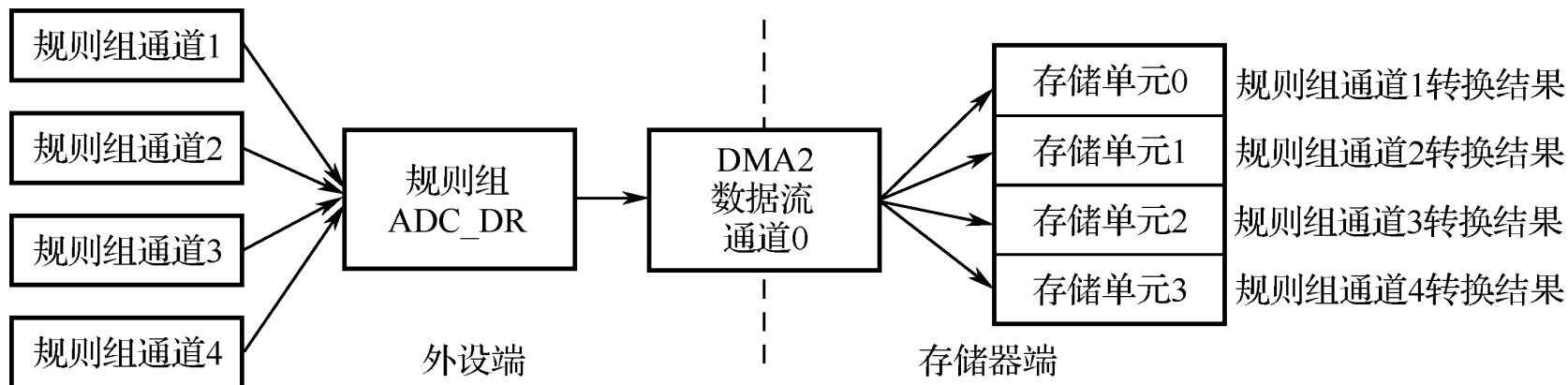
第四次触发：转换的序列0、1、2。

4. DMA控制

DMA (Direct Memory Access, 直接存储器访问) 是STM32内部专门负责搬运数据的硬件模块。它可以在外设和存储器之间自动传输数据，传输过程中不需要CPU逐个读取和写入。

由于规则组通道只有一个ADC_DR，因此，对于**多个规则组通道的转换**，使用**DMA非常有帮助**。

例如，ADC1规则组转换4个输入通道信号时，需要用到DMA2的数据流0的通道0，在扫描模式下，在每个输入通道被转化结束后，都会触发DMA控制器将转换结果从规则组ADC_DR中的数据传输到定义的存储器中。



04 应用步骤及常用库函数

ADC典型应用步骤

以ADC1应用为例。

① 开启PA口时钟和ADC1时钟，设置PA1为模拟输入。

```
RCC_AHB1PeriphClockCmd (RCC_AHB1Periph_GPIOA, ENABLE);  
RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);  
GPIO_Init();
```

② 初始化模拟信号输入的GPIO引脚为模拟方式：

```
GPIO_Init();
```

③ 复位ADC1。

```
ADC_DeInit(ADC1);
```

④ 初始化ADC_CCR寄存器。ADC通用控制寄存器。

```
ADC_CommonInit();
```

⑤ 初始化ADC1参数，设置ADC1的工作模式以及规则序列的相关信息。

```
void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct);
```

ADC典型应用步骤

以ADC1应用为例。

⑥配置规则通道参数:

```
ADC_RegularChannelConfig();
```

⑦如果要用中断, 使能ADC中断, 配置ADC的NVIC。

使能ADC1的转换结束中断:

```
ADC_ITConfig(ADC1, ADC_IT_EOC, ENABLE);
```

配置ADC的NVIC:

```
NVIC_Init();
```

⑧使能ADC

```
ADC_Cmd(ADC1, ENABLE);
```

⑨开启软件转换: *ADC_SoftwareStartConvCmd(ADC1);*

⑩等待转换完成, 读取ADC值

```
ADC_GetConversionValue(ADC1);
```

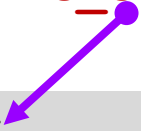
⑪中断服务程序

```
void ADC_IRQHandler(void);
```

常用库函数

1、ADC通用初始化函数

```
void ADC_CommonInit(ADC_CommonInitTypeDef* ADC_CommonInitStruct);
```



```
typedef struct  
  
    uint32_t ADC_Mode;           //多重ADC模式选择  
    uint32_t ADC_Prescaler;      //ADC预分频  
    uint32_t ADC_DMAAccessMode;  //DMA访问模式  
    uint32_t ADC_TwoSamplingDelay; //2个采样阶段之间的延迟  
}ADC_CommonInitTypeDef;
```

这些参数用来配置**ADC_CCR**寄存器的相关参数

常用库函数

2、ADC初始化函数ADC_Init

ADC1
ADC2
ADC3

```
void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct);
```

```
typedef struct
{
    uint32_t ADC_Resolution;           //ADC分辨率
    FunctionalState ADC_ScanConvMode;  //是否使用扫描模式。ADC_CR1位8: SCAN位
    FunctionalState ADC_ContinuousConvMode; //单次转换OR连续转换: ADC_CR2的位1: CONT
    uint32_t ADC_ExternalTrigConvEdge; //外部触发使能方式: ADC_CR2的位29:28, EXTEN
    uint32_t ADC_ExternalTrigConv;     //触发方式: ADC_CR2的位[19:17] : EXTSEL[2:0]
    uint32_t ADC_DataAlign;            //对齐方式: 左对齐还是右对齐: ADC_CR2的位11: ALIGN
    uint8_t ADC_NbrOfChannel;          //规则通道序列长度: ADC_SQR1的位[23:20]: L[3:0]
}ADC_InitTypeDef;
```

常用库函数

2、ADC初始化函数ADC_Init

```
void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct);
```

例:

```
ADC_InitStructure.ADC_Resolution = ADC_Resolution_12b; //12位模式
ADC_InitStructure.ADC_ScanConvMode = DISABLE;           //非扫描模式
ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;    //关闭连续转换
ADC_InitStructure.ADC_ExternalTrigConvEdge
    = ADC_ExternalTrigConvEdge_None; //禁止外部触发检测, 使用软件触发
ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right; //右对齐
ADC_InitStructure.ADC_NbrOfConversion = 1;             //1个转换在规则序列中

ADC_Init(ADC1, &ADC_InitStructure); //ADC初始化
```

常用库函数

3、ADC使能函数

```
void ADC_Cmd(ADC_TypeDef* ADCx, FunctionalState NewState);
```

```
ADC_Cmd(ADC1, ENABLE); //使能指定的ADC1
```

4、ADC使能软件转换函数

```
void ADC_SoftwareStartConvCmd(ADC_TypeDef* ADCx,  
  
FunctionalState NewState)
```

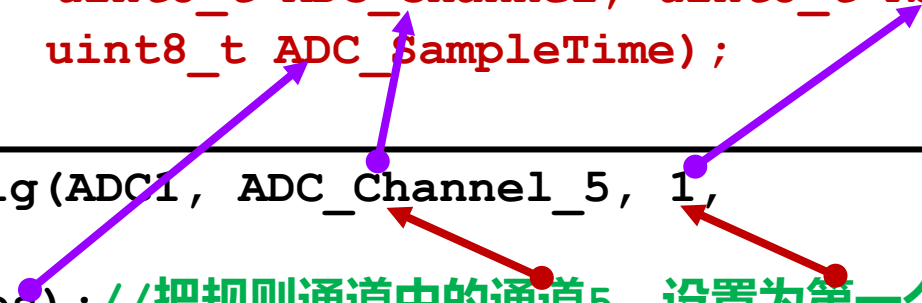
```
ADC_SoftwareStartConvCmd(ADC1, ENABLE); //使能ADC1的软件转换启动
```

常用库函数

5、ADC 规则通道配置函数

```
void ADC_RegularChannelConfig(ADC_TypeDef* ADCx,  
                               uint8_t ADC_Channel, uint8_t Rank,  
                               uint8_t ADC_SampleTime);
```

```
ADC_RegularChannelConfig(ADC1, ADC_Channel_5, 1,  
ADC_SampleTime_480Cycles); //把规则通道中的通道5, 设置为第一个转换
```



6、ADC 获取转换结果函数ADC_GetConversionValue

```
uint16_t ADC_GetConversionValue(ADC_TypeDef* ADCx);
```

```
ADC_GetConversionValue(ADC1); //获取ADC1转换结果
```

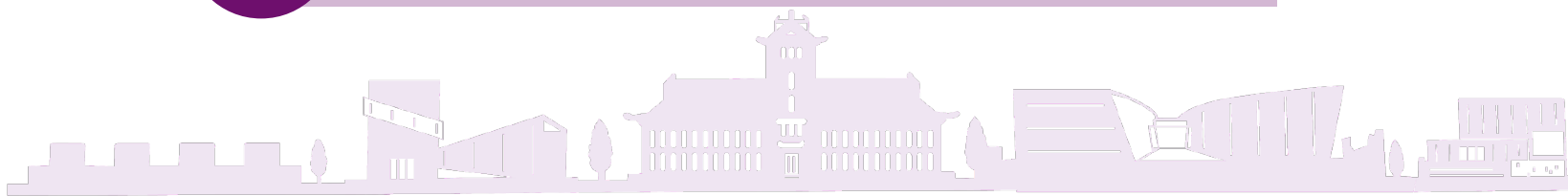
本章小结

01 ADC简介

02 STM32的ADC结构

03 STM32的ADC功能

04 应用步骤及常用库函数



谢谢!